Looped Flows & Flow Map Language Models

Cải thiện Suy luận Lặp và Mô hình hóa Ngôn ngữ Đơn bước

Hung Manh

October 21, 2026

Nội dung trình bày

1. Tóm tắt
2. Giới thiệu
3. Kiến thức nền tảng
4. Phương pháp luận
5. Nghiên cứu liên quan
6. Thử nghiệm và Đánh giá
7. Kết luận
8. Embedded Language Flows (ELF)
9. Flow Map Language Models (FMLM)
10. Shared Memory in Looped Transformers (LPT)

Tóm tắt

- **Vấn đề:** Giải bài toán khó cần nhiều compute time. Các looped models thực hiện điều này qua việc cập nhật tuần hoàn hidden state. Tuy nhiên, huấn luyện gradient ngắn hạn khiến việc học các early updates gặp khó khăn.
- **Đề xuất: Looped flows** - huấn luyện recurrency với các mục tiêu khử nhiễu cục bộ.
- **Cơ chế:**
  - *Temporal alignment*: Giảm dần mức độ nhiễu và sử dụng shared noise.
  - *Inference*: Tích phân vận tốc của probability flow kết hợp với các looped states.
- **Kết quả:** Vượt qua các SOTA looped models, đạt **58.8%** trên ARC-AGI-1 và **12.2%** trên ARC-AGI-2; cho phép scale tính toán và tạo multiple valid predictions.

1. Giới thiệu: Bối cảnh & Thách thức

Bối cảnh & Động lực:

- Giải bài toán khó đòi hỏi gia tăng tính toán lúc inference.
- Khác với Autoregressive LLMs, các looped models sử dụng distributed state và cập nhật tuần hoàn để tích lũy computation.

Thách thức của Looped Models:

- Huấn luyện bằng full BPTT tốn kém bộ nhớ và dễ gặp hiện tượng vanishing/exploding gradients.
- Thực tế chỉ backprop qua vài bước $\rightarrow$ early updates không nhận được tín hiệu để hỗ trợ future computation.
- Hậu quả: Chuỗi lặp không ổn định, sinh ra các failure modes ở cả bài toán đơn giản.

1. Giới thiệu: Giải pháp Looped Flows

Đóng góp chính của Looped Flows:

- Học một chuỗi mục tiêu khử nhiễu cục bộ theo thời gian ở nhiều mức nhiễu khác nhau.
- **Temporal alignment:** Phân bổ huấn luyện theo thời gian, giảm dần mức nhiễu, sử dụng shared noise $\rightarrow$ hidden states duy trì tính hữu ích xuyên suốt.
- **Cơ chế Inference:** Giải bài toán bằng cách tích phân trường vận tốc của probability flow kết hợp với recurrent states.
- **Lợi ích:**
  - Scale compute qua một finer time grid.
  - Đưa ra nhiều dự đoán hợp lệ bằng stochastic/Euler integration.
  - Học được stable recurrency, khắc phục điểm yếu của looped models cũ.

2. Kiến thức nền: Looped Models

- **Định nghĩa:** Cập nhật hidden state $z \in \mathbb{R}^{d_z}$ qua hàm f:

$$z_i = f(z_{i-1}; c)$$

- Đưa ra dự đoán qua đầu đọc g: $\hat{x}_i = g(z_i)$.
- **Hàm loss thực tế:** Để tránh full BPTT, mô hình dùng toán tử dừng gradient $sg(\cdot)$:

$$L_{loop}(f, g) := \mathbb{E}_{c,x}\left[\sum_i CE(\hat{x}_i, x)\right] \quad \text{với } z_i = f(sg(z_{i-1}); c)$$

- *Điểm yếu:* Tổn thất tại bước $\geq i$ không truyền tín hiệu về quá khứ $z_{<i}$. Quá trình lặp deterministic chỉ cho phép 1 lời giải duy nhất.

2. Kiến thức nền: Flow and Diffusion Models

- **Continuous flow matching:** Đi từ nhiễu $x_0 \sim \mathcal{N}(0, \sigma^2 I)$ đến dữ liệu thật $x_1 \sim p_{data}$.
- **Probability path:** Nội suy $I_t := (1-t)x_0 + tx_1$.
- **Velocity field:** $\dot{x}_t = b_t(x_t; c)$.
- Tích phân số (VD: Euler tiến) để xấp xỉ lời giải:

$$x_{t_{i+1}} = x_{t_i} + (t_{i+1} - t_i)\hat{b}_{t_i}(x_{t_i}; c)$$

- **Denoising function (D_t):** Thay vì dự đoán trực tiếp vận tốc, học hàm khử nhiễu độc lập tại mỗi t:

$$L_{flow}(\hat{D}) := \mathbb{E}[CE(\hat{x}_t, x_1)]$$

- *Nhược điểm:* Khó học các tính toán tuần tự do phụ thuộc hoàn toàn vào flow.

3. Looped Flows: Cơ chế Huấn luyện

Stateful Denoiser:

- Khử nhiễu x_{t_i} đồng thời cập nhật looped state z_{t_i}:

$$(\hat{x}_{t_i}, z_{t_i+1}) = \hat{D}_{t_i}(I_{t_i}, sg(z_{t_i}); c)$$

Temporal Alignment:

- Sắp xếp lưới thời gian $t_0 < t_1 < \cdots < t_k$ để mức nhiễu giảm dần.
- **Shared noise:** Mẫu x_0 dùng chung cho toàn bộ quỹ đạo $\rightarrow$ ép mô hình liên kết các thời điểm, giúp z_{t_i} mang thông tin hữu ích cho tương lai.

Adaptive Computation Time (ACT):

- Dùng đầu đọc phân loại nhị phân $q(z_{t_i+1})$ để quyết định dừng suy luận sớm khi độ chính xác đã bão hòa.

3. Looped Flows: Cơ chế Suy luận

- **Stochastic Integration:**
  - Thêm hệ số ngẫu nhiên $\gamma \geq 0$.
  - Lùi nhẹ x_{t_i} về thời điểm $s < t_i$ (tiêm nhiễu mới) trước khi thực hiện bước Euler tiến.
  - Khi $\gamma = 0$, mô hình quay về Euler ODE truyền thống.
- **Inference-time Scaling:**
  - Giải bài toán bằng finer time grid so với lúc huấn luyện.
  - Sử dụng kỹ thuật **best-Q ensembling**: Chạy nhiều mẫu, chọn dự đoán có xác suất thành công cao nhất từ đầu đọc ACT q.

4. Nghiên cứu liên quan

- **Looped models (TRM, FPRM):** Thiếu giám sát toàn cục do cắt gradient, dễ rơi vào spurious attractors. *Looped flows giải quyết bằng cách khởi tạo lặp từ local denoising.*
- **Stochastic looped models:** Tận dụng nhiễu nhưng vẫn phụ thuộc gradient cục bộ. *Looped flows kết hợp lợi ích ngẫu nhiên và giải quyết triệt để học cục bộ.*
- **Flow & Diffusion Models:** Học khử nhiễu độc lập. *Looped flows dùng local objectives tạo lộ trình học mượt mà cho hidden state.*
- **Energy models:** *Looped flows tương đương việc học chuỗi năng lượng làm mịn giảm dần*, giúp quá trình tối ưu dễ dàng hơn.

5. Thử nghiệm: Chính xác & Khả năng mở rộng

So sánh đơn quỹ đạo (Single-trajectory):

- Looped flows áp đảo TRM, FPRM, GRAM trên:
  - Sudoku-Extreme (**97.9%**)
  - Maze-Hard (**86.7%**)
  - ARC-AGI-1 (**58.8%** - tăng từ 44.6% của TRM)
  - ARC-AGI-2 (**12.2%** - tăng từ 7.8% của TRM)

Inference-time scaling:

- Ensembling cải thiện thêm hiệu năng (Sudoku 99.3%, ARC 59.5%).
- Finer time grid tăng accuracy mạnh: Sudoku từ 74.5% (8 steps) $\rightarrow$ 97.9% (128 steps).

Độ ổn định:

- Giải quyết 90.9% các trường hợp thất bại của TRM (bao gồm non-convergence và spurious attractors).

5. Thử nghiệm: Đa dạng Lời giải & Ablation Study

Đa dạng lời giải (Diversity): Đánh giá trên bài toán đa nghiệm.

- N-Queens (8x8): **99.9%** ACC / **91.4%** Coverage.
- Graph Coloring (8 đỉnh): **0.7** conflicts / **89.4%** Coverage.
- Vượt trội hoàn toàn về khả năng tìm ra tập lời giải đa dạng nhờ probability transport.

Ablation Study:

- **Flow:** Bỏ time conditioning, bỏ interpolant, không giảm nhiễu, hay không chia sẻ nhiễu đều làm giảm mạnh hiệu năng.
- **Recurrence:** So với mô hình không lặp (FLM) hoặc self-conditioning, Looped flows đạt khả năng tổng quát hóa tốt nhất trên test set.
- Tích phân SDE ($\gamma = 5$) cho kết quả tốt hơn Euler ODE ($\gamma = 0$).

6. Kết luận

- **Looped Flows** là một framework thanh lịch, giải quyết vấn đề huấn luyện của Looped Models thông qua các mục tiêu khử nhiễu cục bộ.
- **Temporal alignment** giúp trạng thái lặp xây dựng computation hữu ích trên toàn cục, khắc phục hoàn toàn điểm yếu của gradient ngắn hạn.
- Mở ra khả năng **inference-time scaling** mạnh mẽ qua lưới thời gian mịn và tích phân ngẫu nhiên.
- **Hướng tương lai:** Phát triển các thuật toán huấn luyện không cần mô phỏng (simulation-free training).

ELF: Tổng quan bài báo

- **Tên bài báo:** *Embedded Language Flows (ELF)*
- **Ý tưởng cốt lõi:** Mô hình khuếch tán ngôn ngữ liên tục dựa trên Flow Matching trong không gian nhúng biểu diễn (latent space).
- **Đóng góp chính:**
  - Chứng minh continuous diffusion hoàn toàn vượt trội discrete diffusion trên ngôn ngữ nếu khử nhiễu hoàn toàn trong không gian nhúng liên tục.
  - Chỉ thực hiện rời rạc hóa (discretization/decoding) tại bước cuối cùng ($t = 1$).
  - Sử dụng mạng chia sẻ trọng số giữa denoising và decoding, tích hợp sẵn Classifier-Free Guidance (CFG) lúc huấn luyện.

1. ELF: Bối cảnh & Phân loại DLMs

Bối cảnh:

- Diffusion và flow-based models là tiêu chuẩn vàng trong sinh ảnh/video, đang được mở rộng sang xử lý ngôn ngữ.

Phân loại mô hình khuếch tán ngôn ngữ (DLMs):

- *Rời rạc (Discrete):* Thao tác trực tiếp trên token (trước đây được ưu tiên do dễ tối ưu hóa).
- *Liên tục (Continuous):* Ánh xạ token sang không gian biểu diễn liên tục để tiến hành khử nhiễu.

Giải pháp đề xuất của ELF:

- Khử nhiễu hoàn toàn trong không gian nhúng liên tục.
- Định nghĩa trường vận tốc qua thời gian liên tục với Flow Matching.
- Dùng mạng chia sẻ trọng số, chỉ rời rạc hóa tại bước $t = 1$.

2. ELF: Khung làm việc (Framework)

- **Từ rời rạc sang liên tục:** Chuỗi token $s \in \mathcal{V}^L$ được chuyển sang biểu diễn liên tục x bằng bộ mã hóa T5 đông cứng.
- **Nội suy tuyến tính (Rectified Flows):** Tạo biến ẩn z_t dọc theo đường đi:

$$z_t = tx + (1-t)\epsilon, \quad t \in [0,1], \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

- **Trường vận tốc dòng (Flow Velocity):** Đạo hàm theo thời gian:

$$v = \frac{dz}{dt} = x - \epsilon$$

- **Dự đoán x (x-prediction):** Trực tiếp dự đoán biểu diễn sạch $x_\theta(z_t, t)$ thay vì dự đoán vận tốc v, giúp mạng nơ-ron hoạt động ổn định ở số chiều cực cao.

3. ELF: Hàm mục tiêu & Rời rạc hóa

- **Nhánh khử nhiễu (Denoising - chiếm 80% trọng số):** Tối thiểu hóa sai số MSE của trường vận tốc dự đoán:

$$\mathcal{L}_{\text{MSE}} = \mathbb{E}_{t,x,\epsilon}\left[\frac{1}{(1-t)^2}\|x_\theta(z_t,t) - x\|^2\right]$$

- **Nhánh giải mã (Decoding - chiếm 20% trọng số):**
  - Tại bước cuối $t = 1$, biến đổi z_t thành phân phối token qua ma trận unembedding W.
  - Dùng chung trọng số với mạng khử nhiễu, huấn luyện bằng Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = \mathbb{E}_{\tilde{z}}\left[\text{CrossEntropy}(Wx_\theta(\tilde{z}), s)\right]$$

4. ELF: Điều kiện hóa & Dẫn hướng (Guidance)

- **Self-conditioning:** Thực hiện lượt truyền thứ hai điều kiện hóa trên dự đoán trung gian $\hat{x}'$ từ bước trước để tăng độ mượt của quỹ đạo.
- **Classifier-Free Guidance (CFG):** Dễ dàng áp dụng trong miền liên tục:

$$v_{\text{cfg}}(z_t \mid c) = \omega v(z_t \mid c) + (1 - \omega) v(z_t \mid \emptyset)$$

- **Training-time CFG:** Tích hợp thẳng CFG vào hàm mục tiêu vận tốc ngay từ lúc huấn luyện $\rightarrow$ loại bỏ hoàn toàn chi phí tính toán kép khi suy luận.
- **Sinh có điều kiện:** Ghép prompt vào trước chuỗi mục tiêu; phần prompt được giữ nguyên không làm nhiễu.

5. ELF: Kết quả thực nghiệm

Sinh không điều kiện (OpenWebText):

- Ở 32 bước lấy mẫu, ELF-B (105M) đạt **Gen. PPL 24.08**, vượt trội so với các đối thủ rời rạc như MDLM (104.29) và Duo (62.47).
- **Hiệu quả dữ liệu cực cao:** Đạt đỉnh chỉ với 45B token huấn luyện (ít hơn $\sim$10 lần so với > 500B tokens của các baseline).

Chưng cất lũy tiến (Progressive Distillation):

- Cho phép giảm số bước lấy mẫu xuống mức cực ít (**1 đến 4 bước**) mà vẫn duy trì chất lượng sinh văn bản xuất sắc.

Sinh có điều kiện:

- **Tóm tắt XSum:** ROUGE-1 đạt **36.0** (đánh bại các mô hình tự hồi quy cùng kích cỡ).
- **Dịch máy WMT14:** Đạt BLEU **26.4**.

6. ELF: Kết luận & Đóng góp

- **Khẳng định:** Mô hình khuếch tán ngôn ngữ liên tục hoàn toàn có thể cạnh tranh và vượt trội hơn các phương pháp rời rạc.
- **Bí quyết tối ưu của ELF:**
  1. Giữ toàn bộ quỹ đạo làm nhiễu trong không gian nhúng liên tục.
  2. Chỉ thực hiện rời rạc hóa ở bước thời gian cuối cùng ($t = 1$).
  3. Sử dụng mạng chia sẻ trọng số, loại bỏ bộ giải mã phụ trợ rườm rà.
- **Tác động:** Mở ra tiềm năng tạo văn bản chất lượng cao với chi phí huấn luyện thấp và tốc độ suy luận cực nhanh.

FMLM: Tổng quan bài báo

- **Tên bài báo:** *Flow Map Language Models: One-step Language Modeling via Continuous Denoising*
- **Tác giả & Tổ chức:** Chanhyuk Lee, Nicholas M. Boffi, Jinwoo Kim et al. (KAIST & CMU, arXiv:2602.16813)
- **Ý tưởng cốt lõi:** Mô hình hóa ngôn ngữ bằng continuous flows trên biểu diễn one-hot unconstrained, chưng cất thành công mô hình sinh ngôn ngữ **1 bước (one-step)** thông qua toán tử **Two-time Denoiser** trên simplex.
- **Đóng góp nổi bật:**
  - **Phá vỡ định kiến:** Bác bỏ giả thuyết dữ liệu rời rạc bắt buộc phải dùng discrete noising; continuous flow vượt trội cả về chất lượng lẫn tốc độ.
  - **Đột phá sinh 1 bước:** FMLM sinh 1 bước đạt chất lượng vượt qua discrete diffusion 8 bước, tăng tốc **8.3**$\times$.
  - **Two-time Denoiser ($\delta_{s,t}$):** Đưa flow map về nằm trọn trên simplex xác suất, mở đường cho chưng cất bằng semigroup condition và cross-entropy.

1. FMLM: Tử huyệt Factorization Error của Discrete Diffusion

Nút thắt của Autoregressive LLMs:

- Sinh tuần tự từng token một ($O(L)$ bước), tạo ra điểm nghẽn độ trễ khi sinh chuỗi dài.

Tử huyệt Factorization Error của Discrete Diffusion:

- Không gian trạng thái $\mathcal{V}^L$ quá lớn $\rightarrow$ buộc phải dùng xấp xỉ phân rã tích:

$$\tilde{p}_{t|s}(\mathbf{y}_t|\mathbf{y}_s) := \prod_{l=1}^{L} p^l_{t|s}(y^l_t|\mathbf{y}_s)$$

- Giả định các token độc lập có điều kiện chỉ đúng trong giới hạn vô cùng bé ($t \rightarrow s$).
- *Hậu quả ở ít bước ($1 \sim 4$ bước):* Bước nhảy lớn làm mất liên kết ngữ nghĩa giữa các token $\rightarrow$ văn bản vô nghĩa (PPL > 1000) hoặc lặp từ liên tục (entropy sụp đổ < 4.0).

Mệnh đề 3.5 (Giới hạn toán học căn bản):

- Trong không gian rời rạc hữu hạn, **không thể tồn tại sample level**

2. FMLM: Mô hình cơ sở Flow LM (FLM)

- **Biểu diễn One-hot Unconstrained:** $f(\mathbf{y}) \in \mathbb{R}^{L\times|\mathcal{V}|}$, giải mã bằng $g(\mathbf{x}) = \text{argmax}(\mathbf{x})$.
- **Stochastic Interpolant:** Nội suy đường đi giữa Gaussian noise $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$ và one-hot data $\mathbf{x}_1$:

$$I_t := (1-t)\mathbf{x}_0 + t\mathbf{x}_1, \quad t \in [0,1]$$

- **Tránh nút thắt rank bottleneck:** Học vận tốc $\mathbf{b}_t = \mathbb{E}[\mathbf{x}_1 - \mathbf{x}_0 | I_t]$ bằng MSE bị underfitting nghiêm trọng do $\mathbf{x}_0$ full-rank.
- **Chuyển sang Denoiser trên Simplex (Bổ đề 3.1 & Mệnh đề 3.2):**

$$D_t(\mathbf{x}) := \mathbb{E}[\mathbf{x}_1 | I_t = \mathbf{x}] \in \Delta^{|\mathcal{V}|-1}, \quad \mathbf{b}_t(\mathbf{x}) = \frac{D_t(\mathbf{x}) - \mathbf{x}}{1-t}$$

  - D_t là phân phối hậu nghiệm của token, luôn nằm trên đơn hình xác suất simplex.
  - Huấn luyện bằng **Cross-Entropy loss**, chặn chặt khoảng cách Wasserstein $W_2^2 \leq C\epsilon$.

3. FMLM: Tái tham số hóa thời gian theo Decoding Error ($\tau(t)$)

- **Vấn đề:** Tỷ lệ lỗi giải mã $P_e(t) := \frac{1}{L}\sum_l \mathbb{P}(g^l(\mathbf{x}_t) \neq g^l(\mathbf{x}_1))$ dốc đứng sát $t = 1$.
  - Lấy mẫu đều $t \sim \mathcal{U}[0,1]$ lãng phí tín hiệu ở vùng chưa quyết định token và bỏ sót vùng quyết định then chốt khi $|\mathcal{V}|$ lớn.
- **Đề xuất hàm $\tau(t)$ chuẩn hóa tiến độ giải mã:**

$$\tau(t) = 1 - \frac{|\mathcal{V}|}{|\mathcal{V}| - 1} P_e(t)$$

- **Lợi ích thực tế:**
  - Huấn luyện và lấy mẫu đều trong miền $\tau(t) \sim \mathcal{U}[0,1]$.
  - Mỗi bước tính toán đóng góp đồng đều vào việc giảm decoding error.
  - Cài đặt $O(1)$ qua bảng tra cứu trước (LUT) kết hợp nội suy cubic spline, mở rộng thành công lên từ vựng $|\mathcal{V}| \approx 50.000$.

4. FMLM: Two-time Denoiser trên Simplex

Mục tiêu Flow Map:

- Ánh xạ trực tiếp giữa hai thời điểm bất kỳ: $X_{s,t}(\mathbf{x}) = \mathbf{x} + (t-s)\mathbf{v}_{s,t}(\mathbf{x})$.
- Cho phép sinh 1 bước qua $\hat{\mathbf{x}}_1 = X_{0,1}(\mathbf{x}_0)$.
- *Thách thức:* $X_{s,t}$ văng ra khỏi simplex do nhiễu Gaussian $\rightarrow$ dùng MSE cho chất lượng kém.

Định nghĩa Two-time Denoiser ($\delta_{s,t}$):

$$\delta_{s,t}(\mathbf{x}) := \mathbf{x} + (1-s)\mathbf{v}_{s,t}(\mathbf{x})$$

4 Tính chất cốt lõi (Mệnh đề 3.3):

1. **Phục hồi Flow Map:** $X_{s,t}(\mathbf{x}) = \frac{1-t}{1-s}\mathbf{x} + \frac{t-s}{1-s}\delta_{s,t}(\mathbf{x})$.
2. **Bảo toàn trên simplex:** $\delta_{s,t}(\mathbf{x})^l \in \Delta^{|\mathcal{V}|-1} \rightarrow$ dùng được tầng softmax.
3. **Khớp với Denoiser ở đường chéo:** $\delta_{s,s}(\mathbf{x}) = D_s(\mathbf{x})$.
4. **Thỏa mãn tính chất nửa nhóm dạng tổ hợp lồi:**

$$\delta_{s,t} = \gamma\delta_{s,u} + (1-\gamma)\delta_{u,t}(X_{s,u}), \quad \gamma = \frac{(1-t)(u-s)}{(1-u)(t-s)} \in [0,1]$$

5. FMLM: Chưng cất Semigroup & Suy luận 1 bước

Hàm mất mát Semigroup Distillation (Mệnh đề 3.4):

- Tận dụng tính chất tổ hợp lồi trên simplex, tối ưu qua KL divergence với giáo viên FLM $\hat{D}_t$ đóng băng:

$$\mathcal{L}_{\mathrm{KL}}(\hat{\delta}) = \mathbb{E}_{s,u,t}\left[\sum_{l=1}^{L} \mathrm{KL}(\bar{\delta}^{l}_{s,t} \parallel \hat{\delta}^{l}_{s,t}(I_s))\right] + \mathbb{E}_{t}\left[\sum_{l=1}^{L} \mathrm{KL}(\hat{D}_t(I_t)^{l} \parallel \hat{\delta}^{l}_{t,t}(I_t))\right]$$

với mục tiêu giáo viên nửa nhóm: $\bar{\delta}_{s,t} := sg\left(\gamma\hat{\delta}_{s,u}(I_s) + (1-\gamma)\hat{\delta}_{u,t}(\hat{X}_{s,u})\right)$.

Kỹ thuật lấy mẫu biên (Boundary Sampling):

- Do $\tau(t)$ phẳng gần $t = 0$, cố định xác suất $p = 1/32$ lấy mẫu cặp $(s, t) = (0, 1)$ để bơm gradient mạnh cho bước nhảy 1-step.
- **One-step inference:** $\hat{\mathbf{x}}_1 = \hat{\delta}_{0,1}(\mathbf{x}_0)$ sau đó decode qua argmax.

6. FMLM: Kết quả thực nghiệm sinh 1 bước

Hiệu năng ở chế độ ít bước ($N = 1, 2, 4$ steps):

Bộ dữ liệu / Bước	MDLM+SDTT	MDLM+Di4C	Duo+DCD	Duo+Di4C	FMLM (Our
LM1B - 1 bước	1429.48	1217.10	1224.52	292.94*	**119.34** (Ent: 4
LM1B - 2 bước	602.14	621.59	520.08	247.69*	**110.19** (Ent: 4
LM1B - 4 bước	241.01	247.32	210.88	150.67	**98.76** (Ent: 4
OWT - 1 bước	1260.86	1298.80	5743.29	370.51*	**168.30** (Ent: 5
OWT - 4 bước	339.73	239.27	250.86	154.67	**111.31** (Ent: 5

* Mô hình bị sụp đổ Entropy (< 4.0) do lặp từ vô nghĩa liên tục.

- **Tốc độ & Chất lượng:** FMLM 1 bước đạt chất lượng tương đương mô hình rời rạc chưng cất $8 \sim 16$ bước, tăng tốc **8.3**$\times$.
- **Kiểm chứng Mode Collapse:** Self-BLEU (0.073) sát với Real data (0.047); GPT-4.1 Judge xác nhận độ đa dạng cao (39–42%), không bị lặp mẫu.

7. FMLM: Dẫn hướng trong suy luận (Guidance)

Autoguidance (Nâng cao chất lượng sinh không điều kiện):

- $\hat{\mathbf{b}}_t^{(\text{guided})} = \hat{\mathbf{b}}_t^{(\text{weak})} + \eta(\hat{\mathbf{b}}_t - \hat{\mathbf{b}}_t^{(\text{weak})})$.
- Hoạt động trong không gian Euclid: FLM ổn định với scale η lên đến **100**.
- Tại $\eta = 50$, PPL giảm mạnh từ 96.91 xuống **51.62**.
- Mô hình rời rạc sụp đổ hoàn toàn tại $\eta \geq 10$ do ngoại suy logit nhân bản lỗi phân rã nhân tử.

Reward Guidance (FMRG - Điều hướng theo thuộc tính):

- Nhờ Flow Map $X_{t_n,1}$, mô hình có khả năng **nhìn trước điểm kết thúc (look-ahead) vi phân được**:

$$\mathbf{x}_{t_{n+1}} = X_{t_n,t_{n+1}}(\mathbf{x}_{t_n}) + \lambda \nabla_{\mathbf{x}_{t_n}} r\left(X_{t_n,1}(\mathbf{x}_{t_n})\right)$$

- Reward model r (an toàn, cảm xúc, chủ đề) **chỉ cần huấn luyện trên văn bản sạch**.

8. FMLM: Cấu trúc logic (Sudoku) & Kết luận

Mô hình hóa ràng buộc logic toàn cục (Sudoku 9×9):

- *Unconditional 1-step:* FMLM đạt **9.38%** lời giải hợp lệ (tất cả các baseline rời rạc đều $= 0.00\%$).
- *Conditional 1-step (20 ô gợi ý):* FMLM giải đúng **39.84%** chỉ trong 1 forward pass.

Tổng kết 3 bài học cốt lõi:

1. **Phá vỡ định kiến:** Continuous Flow trên biểu diễn One-hot unconstrained vượt trội hơn hẳn Discrete Diffusion trên dữ liệu rời rạc.
2. **Two-time Denoiser ($\delta_{s,t}$):** Đưa Flow Map về nằm trên đơn hình xác suất, biến bài toán chưng cất thành hàm mất mát Cross-Entropy / Semigroup đơn giản và tối ưu.
3. **One-step Generation:** FMLM là mô hình đầu tiên sinh ngôn ngữ 1 bước thành công, mở ra hướng tăng tốc vượt bậc cho mô hình ngôn ngữ và RL finetuning.

LPT: Tổng quan bài báo

- **Tên bài báo:** *The Surprising Effectiveness of Shared Memory in Looped Transformers*
- **Tác giả & Tổ chức:** Giovanni Monea, Keshav Ramji, Yousef El-Kurdi et al. (Cornell University & IBM Research, arXiv:2610.02383)
- **Ý tưởng cốt lõi:** Huấn luyện Looped Transformers chia sẻ bộ nhớ ngay từ đầu (pre-trained shared memory) thay vì lưu cache riêng cho từng bước lặp.
- **Đóng góp nổi bật:**
  - **Xóa bỏ định kiến:** Cắt giảm bộ nhớ không làm giảm chất lượng mà ngược lại giúp tăng mạnh độ chính xác và giảm Perplexity.
  - **Tiết kiệm bộ nhớ vượt bậc:** Giảm **76–79%** context memory so với Standard Transformer cùng kích thước ($150\text{M} \sim 1\text{B}$).
  - **Đại lộ Gradient (Gradient Highway):** Cơ chế đọc shared memory giúp dòng gradient từ hàm loss bước cuối truyền thẳng về vòng lặp đầu tiên, ổn định đệ quy sâu.

1. LPT: Bối cảnh & ”Cái bẫy” bộ nhớ KV Cache

Nguyên lý Looped Transformers (LT):

- Áp dụng cùng một khối L tầng Transformer lặp lại R lần trên hidden state trước khi dự đoán token:

$$h_{\text{in}}^{(t+1)} = \mathcal{N}_{\text{rec}}\left(h^{(t)} + \mathcal{M}^{L}(h_{\text{in}}^{(t)})\right)$$

- Đạt độ sâu tính toán hiệu dụng $L \cdot R$ nhưng chỉ tốn số lượng tham số của L tầng.

Cái bẫy bộ nhớ ngữ cảnh dài (Context Memory Explosion):

- Trong LT truyền thống, mỗi recursion $t \in [1, R]$ ghi một bảng KV cache riêng biệt.
- Tổng dung lượng cache: $\mathcal{O}(R \cdot L \cdot T \cdot d)$. Tại $T = 4096$, bộ nhớ KV cache phình to vượt xa bộ nhớ lưu trữ trọng số mô hình.
- *Thất bại của giải pháp cũ:* Bỏ bớt cache lúc suy luận (post-hoc inference drop như Ouro, Huginn) khiến chất lượng sinh sụp đổ nghiêm trọng.

2. LPT: Kiến trúc Looped Prediction Transformer

Phân tách Bộ nhớ và Dự đoán theo trục đệ quy:

- **State Recursion ($t = 1$):** Đọc toàn bộ ngữ cảnh, xây dựng và ghi lại bảng full-context KV cache duy nhất $\mathcal{C}^{(1)}$ trên toàn bộ L tầng.
- **Prediction Recursions ($t \geq 2$):** Không ghi thêm full cache, chỉ duy trì một cửa sổ trượt cục bộ siêu ngắn $\omega = 64$ token ($\mathcal{C}^{(t)}_{[i-\omega,i]}$, chiếm $< 2\%$ ngữ cảnh).

Attention đồng nhất qua một phép Softmax duy nhất:

$$\mathcal{A}_i^{(t)} = \mathcal{C}^{(1)}_{1:i} \cup \mathcal{C}^{(t)}_{[i-\omega,i]}$$

- Query $q_i^{(t)}$ chú ý đồng thời vào Shared Memory và cửa sổ cục bộ trong một lượt tính toán (tối ưu hóa qua FlexAttention block mask).
- **Biến thể Hybrid (H-LPT):** Kết hợp các tầng Linear Attention (Gated DeltaNet) với các tầng Full Attention dùng Shared Memory theo tỷ lệ 4:1.

3. LPT: Kết quả thực nghiệm phá vỡ biên giới Quality–Memory

Đánh giá trên FineWeb-Edu (Context $T = 4096$, Quy mô 150M – 1B tham số):

Mô hình	**Recursions (R)**	**Val PPL (↓)**	**Zero-shot Acc**
Standard Transformer (STD)	1	Baseline	Baseline
Vanilla LT (Full Cache)	5	Dừng cải thiện	Thấp hơn LPT 0.4
LPT (Ours)	5	**-1.09 nats** vs LT	**+4.8%** vs LT
H-LPT (Ours)	5	**-1.82 nats** vs STD	**+5.6%** vs ST

- **Quy luật mở rộng theo độ sâu:** LT truyền thống dừng cải thiện sau 3 vòng lặp; LPT tiếp tục tăng trưởng chất lượng mượt mà đến 5 vòng lặp.
- **Huấn luyện đơn giản:** LPT chỉ cần dùng Final Loss đơn giản ở bước cuối, không cần cơ chế Adaptive loss phức tạp như Ouro mà vẫn cực kỳ ổn định.

4. LPT: Giải mã cơ chế 1 – Đại lộ Gradient (Gradient Highway)

Nghịch lý suy giảm gradient trong đệ quy sâu:

- Trong LT chuẩn, gradient từ hàm loss ở bước cuối bị suy giảm nghiêm trọng khi truyền ngược qua nhiều bước lặp tuần tự $\rightarrow$ vòng 1 không nhận đủ tín hiệu.

Shared Memory đóng vai trò Gradient Highway:

- Do query ở các recursion sau đọc trực tiếp Shared KV cache của vòng 1, gradient có một đường dẫn tắt truyền thẳng về vòng 1.
- Chuẩn gradient truyền về vòng 1 trong LPT được phục hồi tương đương Standard Transformer, không bị sụt giảm theo số vòng lặp.

Thí nghiệm triệt tiêu (Ablation):

- Khi ngắt dòng gradient này (*detach shared KV*), mô hình ngay lập tức bị suy giảm chất lượng nặng nề hoặc không thể hội tụ khi tăng số vòng lặp.

5. LPT: Giải mã cơ chế 2 – Phân hóa biểu diễn & Vai trò của ω

Phân hóa không gian biểu diễn (PCA không gian Key):

- Phân tích PCA cho thấy Shared Keys và Local Keys tự động phân tách thành hai cụm không gian hoàn toàn riêng biệt.
- *Shared Keys:* Chuyên biệt hóa lưu trữ ngữ cảnh tĩnh toàn cục.
- *Local Keys:* Chuyên biệt hóa xử lý các tương tác cục bộ ngắn hạn.

Thí nghiệm quét kích thước cửa sổ ω ($0 \to 4096$):

- LPT vượt trội LT chuẩn ở **mọi kích thước cửa sổ** ω.
- Ngay cả khi $\omega = 0$ (hoàn toàn không lưu cache quá khứ ở các vòng sau), LPT vẫn tốt hơn LT giữ full cache.
- Tỷ trọng Attention: Càng về các recursion sau, mô hình càng dồn sự chú ý vào Shared Memory (chiếm tới **73–80%** attention mass).

6. LPT: Đo đạc thực tế trên phần cứng NVIDIA H100

Khắc phục nút thắt Memory-Bound khi phục vụ mô hình:

- **Khả năng chứa Batch (Batch Capacity):** Nhờ cắt giảm 79% context memory, H-LPT cho phép tăng batch size lên gấp **3.6$\times$ – 5.0$\times$** so với Standard Transformer trước khi đầy 80GB VRAM.
- **Thông lượng thực tế (Throughput):**
  - Ở $R = 3$: H-LPT đạt thông lượng cao gấp **1.4 – 1.5$\times$** so với Standard Transformer.
  - Ở $R = 5$: H-LPT duy trì thông lượng **81–88%** so với Standard Transformer, dù thực hiện số phép tính gấp nhiều lần.

Ý nghĩa thực tiễn

Việc tăng batch size trên GPU bù đắp hoàn toàn chi phí tính toán lặp lại, biến Looped Transformers thành giải pháp khả thi trong môi trường sản xuất thực tế.

7. LPT: Kết luận & Định hướng tương lai

- **Inductive Bias mới:** Chia sẻ bộ nhớ không phải là sự đánh đổi chấp nhận mất mát, mà là một cấu trúc kiến trúc vượt trội giúp phân định rõ ràng giữa lưu trữ ngữ cảnh và tinh chỉnh suy luận.
- **Đột phá đệ quy sâu:** Gradient Highway qua shared cache mở đường cho việc mở rộng Looped Transformers lên số vòng lặp lớn mà không gặp hiện tượng vanishing gradient.
- **Hiệu quả triển khai:** Giảm 76–79% bộ nhớ ngữ cảnh, tăng 5$\times$ batch capacity trên GPU hiện đại.

Hướng phát triển tiếp theo:

- Tích hợp cơ chế dừng sớm linh hoạt (adaptive compute / early exit) theo từng độ khó của token.
- Mở rộng sang các bài toán suy luận đa bước phức tạp (complex multi-step reasoning).

Cảm ơn bạn đã theo dõi!

Q&A